



Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

Chapter 6: Backtracking and Branch-and-Bound

Algorithms Analysis and Design (C03031)

Instructor: Assoc. Prof. Dr. Vo Thi Ngoc Chau

Email: chauvtn@hcmut.edu.vn



Course Learning Outcomes

1. Able to analyze the complexity of the algorithms (recursive or iterative) and estimate the efficiency of the algorithms.
2. Improve the ability to design algorithms in different areas *by decomposing a computing problem into advanced algorithm design strategies.*
3. Able to discuss NP-completeness.



References

- [1] Cormen, T. H., Leiserson, C. E, and Rivest, R. L., *Introduction to Algorithms*, The MIT Press, 2009.
- [2] Levitin, A., *Introduction to the Design and Analysis of Algorithms*, 3rd Edition, Pearson, 2012.
- [3] Sedgewick, R., *Algorithms in C++*, Addison-Wesley, 1998.
- [4] Weiss, M.A., *Data Structures and Algorithm Analysis in C*, The Benjamin/Cummings Publishing, 1993.



Course Outline

- ❑ Chapter 1. Fundamentals
- ❑ Chapter 2. Divide-and-Conquer Strategy
- ❑ Chapter 3. Decrease-and-Conquer Strategy
- ❑ Chapter 4. Transform-and-Conquer Strategy
- ❑ Chapter 5. Dynamic Programming and Greedy Strategies
- ❑ **Chapter 6. Backtracking and Branch-and-Bound**
- ❑ Chapter 7. NP-completeness
- ❑ Chapter 8. Approximation Algorithms



Chapter 6.

Backtracking and Branch-and-Bound

- 6.1. Backtracking algorithms
 - 6.1.1. The Knight's Tour problem
 - 6.1.2. The Eight Queens problem
- 6.2. Branch-and-Bound algorithms
 - 6.2.1. Traveling Salesman problem



6.1. Backtracking algorithms

- ❑ The task is to determine an algorithm for finding solutions to specific problems not by following a fixed rule of computation, but by *trial-and-error*.
- ❑ The common pattern is to *decompose* the trial-and-error process into *partial tasks*. Often these tasks are most naturally expressed in recursive terms and consist of the *exploration of a finite number of subtasks*.
- ❑ We may view the entire process as *a search process* that gradually builds up and scans *a tree of subtasks*.



6.1. Backtracking algorithms

□ State space tree

- To illustrate the working of a backtracking algorithm, we construct a tree structure which records the executed selections. This tree structure is called *state space tree* or *search tree*.
- The root node of the tree represents the *original state* before the start of the search process.
- The nodes in the first level represent the selections corresponding to the first component of the solution.
- The nodes in the second level represent the selections corresponding to the second component of the solution, and so on.



6.1. Backtracking algorithms

□ State space tree

- A node in the state space tree is called *promising* if it corresponds to the partial solution which can lead to a complete solution; otherwise, it is called a *non-promising* solution.
- Leaf nodes represent the *dead-end* situations or a *complete solution*.
- If each node in the state space tree has α children on average, and the *length* of the solution path is N , then the total number of nodes in the tree is computed as

$$1 + \alpha^1 + \alpha^2 + \dots + \alpha^N = (\alpha^{N+1} - 1) / (\alpha - 1)$$

- When N is large enough, $(\alpha^{N+1} - 1) / (\alpha - 1)$ is approximately equal to α^N .



6.1. Backtracking algorithms

- A backtracking algorithm with a state space tree
 - In the majority of cases, a state-space tree for a backtracking algorithm is constructed in the manner of *depth-first search*.
 - If the current node is *promising*, its child is generated by adding the first remaining legitimate option for the next component of a solution, and the processing moves to this child.
 - If the current node turns out to be *nonpromising*, the algorithm *backtracks to the node's parent* to consider the next possible option for its last component; if there is no such option, it backtracks one more level up the tree, and so on.
 - Finally, if the algorithm reaches a complete solution to the problem, it either stops (if *just one solution* is required) or continues searching for *other possible solutions*.



6.1. Backtracking algorithms

```
procedure try;  
begin initialize selection of candidates;  
repeat  
    select next;  
    if acceptable then  
        begin  
            record it;  
            if solution incomplete then  
                begin  
                    try next step;  
                    if not successful then cancel recording  
                end  
            end  
        end  
until successful  $\vee$  no more candidates  
end
```

The general
pattern of a
backtracking
algorithm



6.1. Backtracking algorithms

If at each step the number of candidates to be investigated is fixed, say m , then the algorithm is rewritten: $\Rightarrow$

It is invoked by the statement $try(1)$.

```
procedure try (i: integer);  
var k : integer;  
begin k:=0;  
  repeat  
    k:=k+1; select k-th candidate;  
    if acceptable then  
      begin  
        record it;  
        if i<n then // n: length of the solution path  
          begin  
            try (i+1);  
            if not successful then  
              cancel recording  
          end  
        end  
      end  
    until successful  $\vee$  (k=m) // m: #children of each node  
end
```



6.1. Backtracking algorithms

For example: Given the following problem:

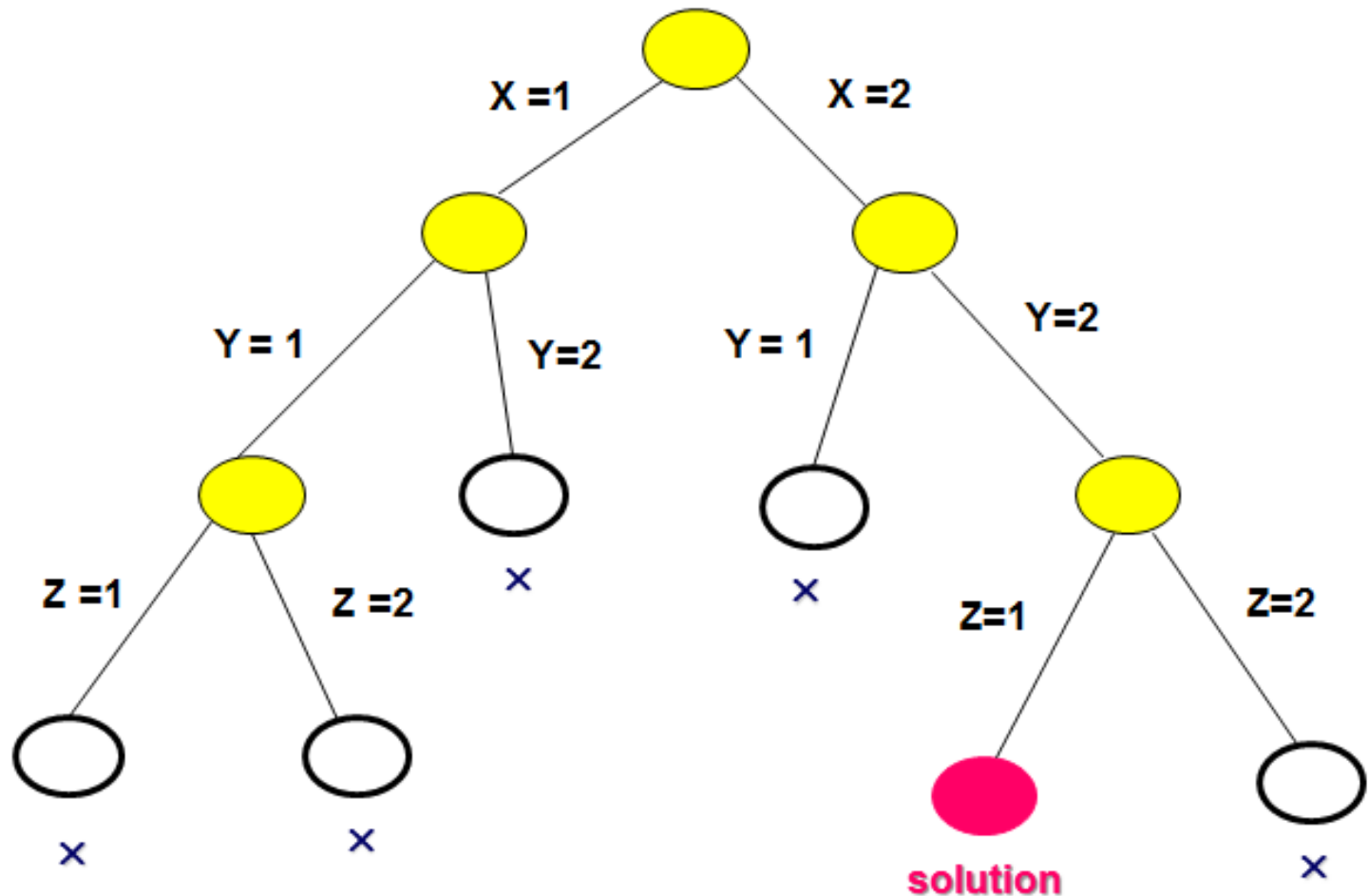
Set of variables: X, Y, Z .

Let's assign values from the set of values $\{1, 2\}$ to the variables such that the assignment satisfies the following constraints: $X = Y, X \neq Z, Y > Z$.

Solve this problem by a backtracking algorithm.
What is a state space tree for the given instance of the problem?

6.1. Backtracking algorithms

Let's assign values from the set of values $\{1,2\}$ to X , Y , and Z for constraints: $X = Y$, $X \neq Z$, $Y > Z$.





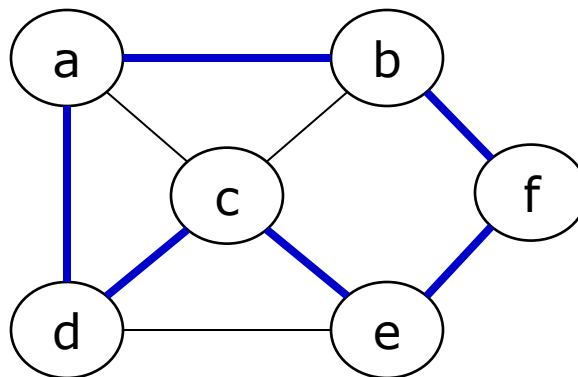
6.1. Backtracking algorithms

- Time complexity of a backtracking algorithm
 - Since time complexity of a backtracking algorithm is proportional to the number of nodes in the state space tree, the time complexity of this algorithm is proportional to α^N .
 - Computational time of a backtracking algorithm is always *exponential*.

6.1. Backtracking algorithms

- The problem of finding a Hamiltonian cycle in an undirected graph
 - Given an undirected graph, find a Hamiltonian cycle which is a path that starts and ends at the same vertex and passes through all the other vertices exactly once.
 - For example: Given an undirected graph (G), a Hamiltonian cycle is in blue.

Graph (G)



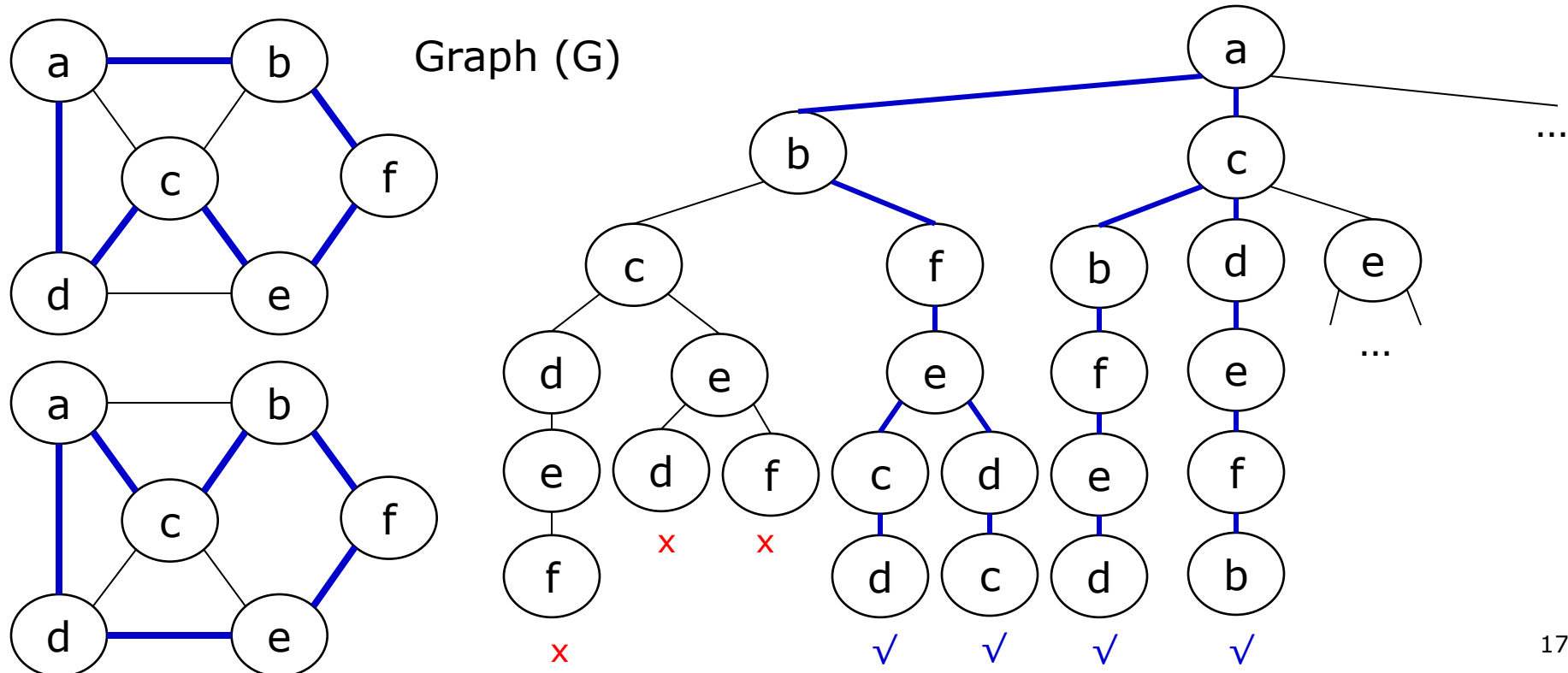


6.1. Backtracking algorithms

- ❑ The problem of finding a Hamiltonian cycle in an undirected graph
 - Given an undirected graph, find a Hamiltonian cycle which is a path that starts and ends at the same vertex and passes through all the other vertices exactly once.
 - *State space tree*
 - ❑ *Root*: a starting vertex to start a depth-first search recursively on the graph
 - ❑ Each node represents the j -th visited vertex forming a path of length $j-1$ of the cycle.
 - ❑ A leaf that represents the V -th visited vertex that has an edge back to the starting vertex forms a solution.
 - ❑ The tree is constructed in the depth-first-search manner.

6.1. Backtracking algorithms

- The problem of finding a Hamiltonian cycle in an undirected graph
 - For example: given an undirected graph (G), one Hamiltonian cycles are found in blue.



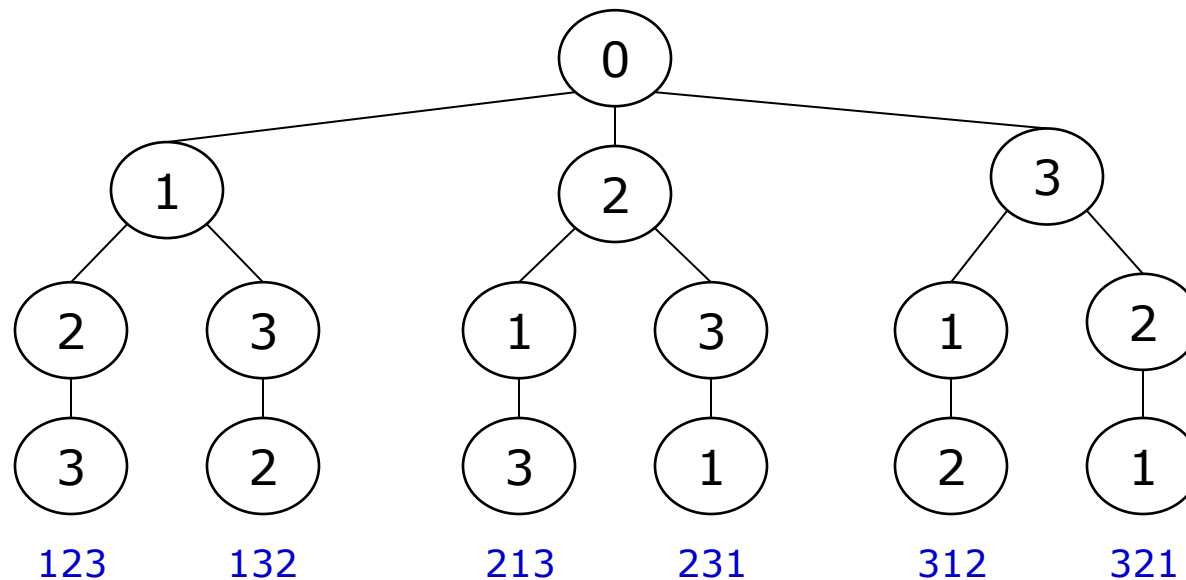


6.1. Backtracking algorithms

- The problem of generating all permutations
 - Given a set of n elements $A = \{a_1, a_2, \dots, a_n\}$, we want to generate all $n!$ different permutations from that set.
 - For example: $A = \{1, 2, 3\}$. All permutations are: 123, 132, 312, 213, 231, 321.
 - *State space tree*
 - *Root*: the starting point with no decision made yet
 - Each node represents a part of one permutation in the solution formed so far.
 - Each leaf represents one permutation in the solution.
 - The tree is constructed in the depth-first-search manner.

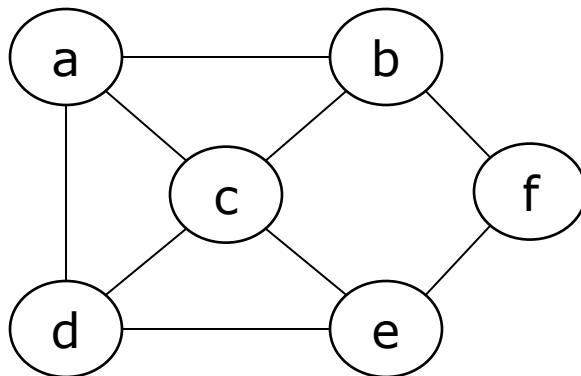
6.1. Backtracking algorithms

- The problem of generating all permutations
 - For example: $A = \{1, 2, 3\}$. All permutations are: 123, 132, 312, 213, 231, 321.

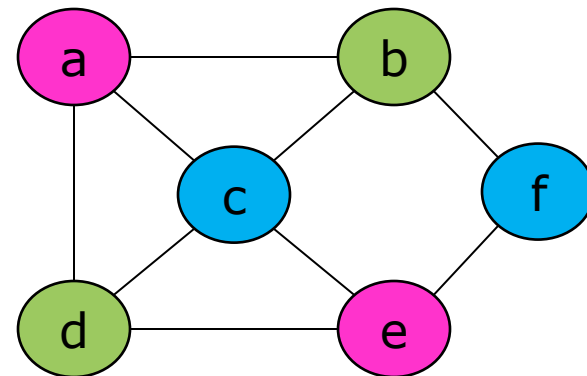


6.1. Backtracking algorithms

- The m -coloring problem with m colors
 - Given a graph, check whether there exists a coloring of the graph's vertices with no more than m colors in such a way that no two adjacent vertices are assigned the same color.
 - For example: given an undirected graph (G), its colored graph with {pink, green, blue} is below.



Graph (G)



Colored graph (G)

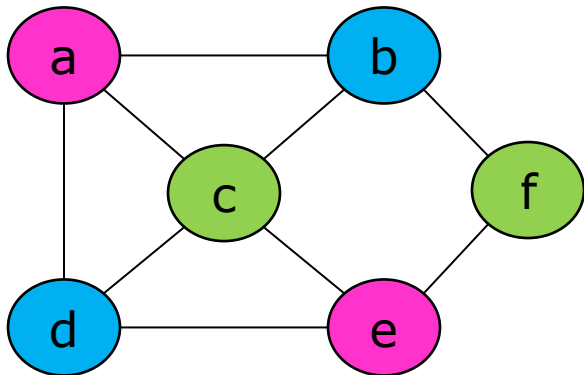


6.1. Backtracking algorithms

- The m -coloring problem with m colors
 - Given a graph, check whether there exists a coloring of the graph's vertices with no more than m colors in such a way that no two adjacent vertices are assigned the same color.
 - *State space tree*
 - *Root*: a starting vertex to be colored
 - Each node represents a vertex which is the j -th colored one.
 - Each edge represents a color to color the starting vertex of this edge.
 - A leaf is a dead-end or a solution. A solution leaf is an end of the V -th colored vertex.
 - A tree is constructed in the depth-first search manner.



- For example: given an undirected graph (G), its colored graph with {pink, green, blue} is below.





6.1. Backtracking algorithms

□ The subset-sum problem

- Find a subset of a given set $S = \{s_1, \dots, s_n\}$ of n positive integers whose sum is equal to a given positive integer d .
- For example: For an ordered set $S = \{3, 5, 6, 7\}$ and $d = 15$, solution = $\{3, 5, 7\}$.

■ *State space tree*

- *Root*: the starting point with no decision made yet.
- Each node represents the current accumulated sum.
- Nonpromising nodes with constraints:

$$s' + s_{i+1} > d \text{ (the sum } s' \text{ is too large)}$$

$$s' + \sum_{j=i+1}^n s_j < d \text{ (the sum } s' \text{ is too small)}$$

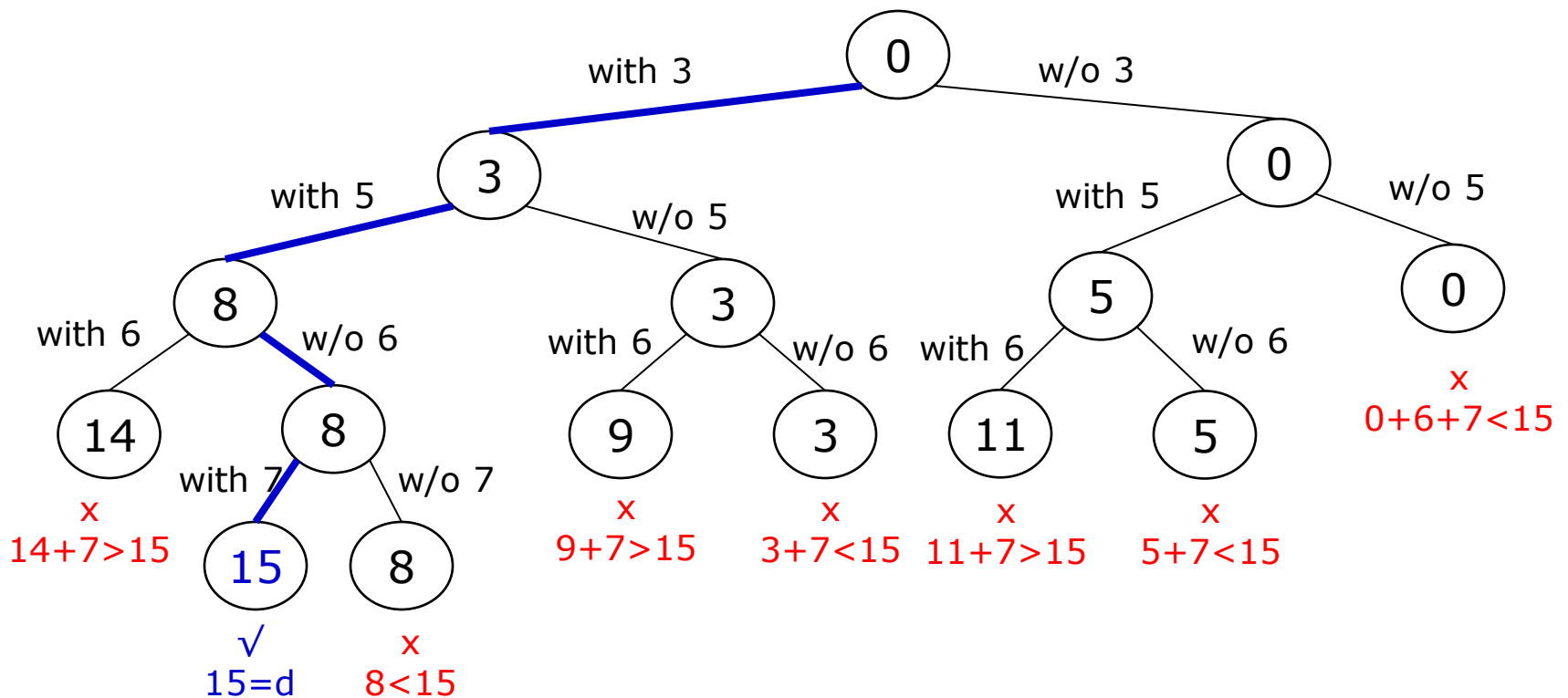
Where: $s_1 \leq s_2 \leq \dots \leq s_n$,

s' is the value of the current node being considered as a nonpromising node. ²³

6.1. Backtracking algorithms

□ The subset-sum problem

- For example: For an ordered set $S = \{3, 5, 6, 7\}$ and $d = 15$, solution = $\{3, 5, 7\}$.





6.1. Backtracking algorithms

Your turn:

- ❑ 6.1.1. List three main characteristics of a backtracking algorithm.
- ❑ 6.1.2. What is a state space tree? Give another name of this tree.
- ❑ 6.1.3. Compare the backtracking strategy with the brute-force one.

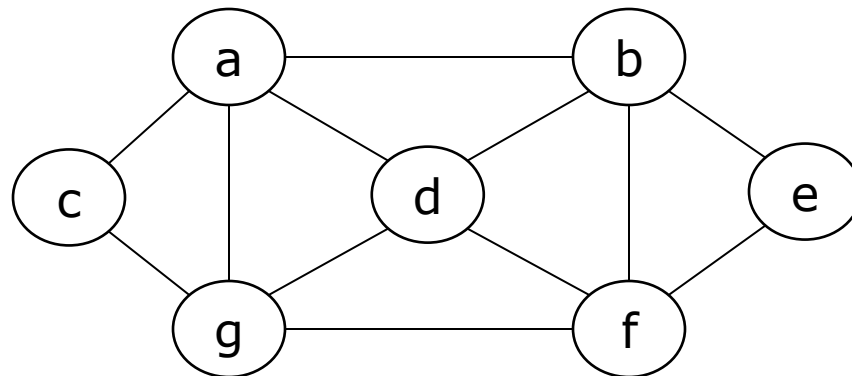


6.1. Backtracking algorithms

Your turn:

- 6.1.4. Draw a state space tree for each instance of the following problems. Design and analyze the backtracking algorithms for these problems.
 - 6.1.4.1. The problem of generating all permutations of $\{1, 2, 3, 4\}$
 - 6.1.4.2. The subset-sum problem: $S = \{3, 1, 7, 4, 5\}$ and $d = 11$
 - 6.1.4.3. The problem of finding a Hamiltonian cycle in (G)
 - 6.1.4.4. The m -coloring problem on (G) with $\{\text{red, green blue}\}$

Graph (G)





6.1.1. The Knight's Tour problem

- ❑ Given a $n \times n$ board with n^2 fields, a knight – being allowed to move according to the rules of chess – is placed on the field with initial coordinates x_0, y_0 .
- ❑ The problem is to find a covering of the entire board, if there exists one, i.e., to compute a tour of $n^2 - 1$ moves such that every field of the board is visited exactly once.
- ❑ The obvious way to reduce the problem of covering n^2 fields is to consider the problem of either
 - performing a next move or
 - finding out that none is possible.
- ❑ Let us define an algorithm trying to perform a next move.



6.1.1. The Knight's Tour problem

- Given a $n \times n$ board with n^2 fields, with initial coordinates x_0, y_0 , the problem is to find a covering of the entire board, if there exists one, i.e., to compute **a tour of $n^2 - 1$ moves** such that every field of the board is visited exactly once.

	1	2	3	4	5
1	1				
2					
3					
4					
5					

	1	2	3	4	5
1	1	6	15	10	21
2	14	9	20	5	16
3	19	2	7	22	11
4	8	13	24	17	4
5	25	18	3	12	23



6.1.1. The Knight's Tour problem

```
procedure try next move;  
begin initialize selection of moves;  
  repeat  
    select next candidate from list of next moves;  
    if acceptable then  
      begin  
        record move;  
        if board not full then  
          begin  
            try next move;  
            if not successful then erase previous recording  
          end  
        end  
      until (move was successful)  $\vee$  (no more candidates)  
    end
```



6.1.1. The Knight's Tour problem

- ❑ In this algorithm, we use a new “problem-solving” method based on *backtracking*.
- ❑ Its main feature is that:

“Steps toward the total solution are attempted and recorded that may later be *taken back* and *erased* in the recordings when it is discovered that the step does not possibly lead to the total solution, that the step leads to a “*dead-end*”.”



6.1.1. The Knight's Tour problem

□ Data representation

- Let's represent the board by a matrix, h .

type index = $1..n$;

var h : array[index, index] of integer;

$h[x, y] = 0$: field $\langle x, y \rangle$ has not been visited

$h[x, y] = i$: field $\langle x, y \rangle$ has been visited in the i -th move ($1 \leq i \leq n^2$)

- The condition “board not full” can be expressed as “ $i < n^2$ ”.
- u, v : coordinates of possible move destination.
- The predicate “acceptable” can be expressed as $(1 \leq u \leq n) \wedge (1 \leq v \leq n) \wedge (h[u, v] = 0)$.



6.1.1. The Knight's Tour problem

```
procedure try(i: integer; x,y : index; var q: boolean);  
var u, v: integer; q1 : boolean;  
begin initialize selection for moves;  
  repeat let u, v be the coordinates of the next move ;  
    q1:= false;  
    if  $(1 \leq u \leq n) \wedge (1 \leq v \leq n) \wedge (h[u,v]=0)$  then  
      begin h[u,v]:=i;  
        if  $i < n*n$  then  
          begin  
            try(i + 1, u, v, q1); if  $\neg q1$  then h[u,v]:=0  
          end  
          else q1:= true  
        end  
      until q1  $\vee$  (no more candidates);  
    q:=q1  
end
```




6.1.1. The Knight's Tour problem

Given the current coordinate pair $\langle x, y \rangle$, there are 8 potential candidates for coordinates $\langle u, v \rangle$ of the destination. They are numbered 1 to 8 as follows:

	3		2	
4				1
		$\oplus$		
5				8
	6		7	



6.1.1. The Knight's Tour problem

Relationship between the current coordinate pair $\langle x, y \rangle$ and 8 possible moves:

	$y = 1$	$y = 2$	$y = 3$	$y = 4$	$y = 5$
$x = 1$		3 $(x-2, y-1)$		2 $(x-2, y+1)$	
$x = 2$	4 $(x-1, y-2)$				1 $(x-1, y+2)$
$x = 3$			$\oplus$ (x, y)		
$x = 4$	5 $(x+1, y-2)$				8 $(x+1, y+2)$
$x = 5$		6 $(x+2, y-1)$		7 $(x+2, y+1)$	



6.1.1. The Knight's Tour problem

A simple method of obtaining u , v from x , y is by addition of the *coordinate differences* stored in two arrays a and b .

Then an index k may be used to number the “next candidate”.

```
program knightstour (output);  
  
const n = 5; nsq = 25;  
  
type index = 1..n  
  
var i,j: index; q: boolean;  
  
s: set of index;  
  
a,b: array [1..8] of integer;  
  
h: array [index, index] of integer;
```



6.1.1. The Knight's Tour problem

A *backtracking* algorithm for the knight's tour problem.

Time complexity:

- Abstract operation: *move*
- *Worst case* analysis
- $C_n = 8^{n^2-1} = O(8^{n^2})$

```
procedure try (i: integer; x, y: index; var q:boolean);  
var k,u,v : integer; q1: boolean;  
begin   k:=0;  
      repeat  
        k:=k+1; q1:=false; u:=x+a[k]; v:=y+b[k];  
        if (u in s)  $\wedge$  (v in s) then  
          if h[u,v]=0 then  
            begin  
              h[u,v]:=i;  
              if i < nsq then  
                begin  
                  try(i+1, u, v, q1);  
                  if  $\neg$  q1 then h[u,v]:=0  
                end  
              else q1:=true  
            end  
          end  
        until q1  $\vee$  (k = 8);  
        q:=q1  
      end
```



6.1.1. The Knight's Tour problem

begin

s:=[1,2,3,4,5];

a[1]:= -1; b[1]:= 2;

a[2]:= -2; b[2]:= 1;

a[3]:= -2; b[3]:= -1;

a[4]:= -1; b[4]:= -2;

a[5]:= 1; b[5]:= -2;

a[6]:= 2; b[6]:= -1;

a[7]:= 2; b[7]:= 1;

a[8]:= 1; b[8]:= 2;

for i:=1 to n do

for j:=1 to n do h[i,j]:=0;

h[1,1]:=1; try (2,1,1,q);

if q then

for i:=1 to n do

begin

for j:=1 to n do

write(h[i,j]:5);

writeln

end

else writeln ('NO SOLUTION')

end.



6.1.1. The Knight's Tour problem

The recursive procedure is initiated by a call with the initial position $\langle x_0, y_0 \rangle$, from which the tour starts.

$h[x_0, y_0] := 1; \text{try}(2, x_0, y_0, q)$

A solution is obtained with initial position $\langle 1, 1 \rangle$ for $n = 5$.

1	6	15	10	21
14	9	20	5	16
19	2	7	22	11
8	13	24	17	4
25	18	3	12	23



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1				
2					
3					
4					
5					



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1				
2					
3		2			
4					
5					



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1				
2				3	
3		2			
4					
5					



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4			
2				3	
3		2			
4					
5					



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4			
2				3	
3	5	2			
4					
5					



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4			
2			6	3	
3	5	2			
4					
5					



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4			7
2			6	3	
3	5	2			
4					
5					



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4			7
2			6	3	
3	5	2		8	
4					
5					



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9		7
2			6	3	
3	5	2		8	
4					
5					



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9		7
2	10		6	3	
3	5	2		8	
4					
5					



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9		7
2	10		6	3	
3	5	2		8	
4		11			
5					



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9		7
2	10		6	3	
3	5	2		8	
4		11			
5				12	



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9		7
2	10		6	3	
3	5	2		8	13
4		11			
5				12	



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10		6	3	
3	5	2		8	13
4		11			
5				12	



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	
3	5	2		8	13
4		11			
5				12	



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	
3	5	2		8	13
4	16	11			
5				12	



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	
3	5	2	17	8	13
4	16	11			
5				12	



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	18
3	5	2	17	8	13
4	16	11			
5				12	



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	18
3	5	2	17	8	13
4	16	11		19	
5				12	



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	18
3	5	2	17	8	13
4	16	11		19	
5		20		12	



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	18
3	5	2	17	8	13
4	16	11		19	
5		20		12	

X



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	18
3	5	2	17	8	13
4	16	11		19	
5				12	

X



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	18
3	5	2	17	8	13
4	16	11			
5				12	

X



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	
3	5	2	17	8	13
4	16	11			
5		18		12	



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	
3	5	2	17	8	13
4	16	11		19	
5		18		12	



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	20
3	5	2	17	8	13
4	16	11		19	
5		18		12	



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	20
3	5	2	17	8	13
4	16	11		19	
5		18		12	

X



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	
3	5	2	17	8	13
4	16	11		19	
5		18		12	

X



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	
3	5	2	17	8	13
4	16	11			
5		18		12	

X



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	
3	5	2	17	8	13
4	16	11			18
5				12	



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	
3	5	2	17	8	13
4	16	11			18
5			19	12	



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	
3	5	2	17	8	13
4	16	11			18
5			19	12	

X



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	
3	5	2	17	8	13
4	16	11			<i>18</i>
5				12	

X



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	
3	5	2	17	8	13
4	16	11			
5				12	

X



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	4	9	14	7
2	10	15	6	3	
3	5	2		8	13
4	16	11			
5			17	12	



6.1.1. The Knight's Tour problem

Demonstration with an initial position $\langle 1, 1 \rangle$ on a 5-by-5 board:

	1	2	3	4	5
1	1	6	15	10	21
2	14	9	20	5	16
3	19	2	7	22	11
4	8	13	24	17	4
5	25	18	3	12	23



6.1.1. The Knight's Tour problem

Your turn:

- ❑ 6.1.1.1. Trace-by-hand the backtracking algorithm for the knight's tour problem on a 4-by-4 board, starting from $\langle 1, 1 \rangle$.
- ❑ 6.1.1.2. Trace-by-hand the backtracking algorithm for the knight's tour problem on a 5-by-5 board, starting from $\langle 2, 2 \rangle$.
- ❑ 6.1.1.3. Trace-by-hand the backtracking algorithm for the knight's tour problem on a 6-by-6 board, starting from $\langle 3, 3 \rangle$.



6.1.1. The Knight's Tour problem

Your turn:

- ❑ 6.1.1.4. Modify the backtracking algorithm previously designed for one solution to a given instance of the knight's tour problem so that it can generate all the possible tours starting from a given initial position. Analyze the time complexity of the modified algorithm.
- ❑ 6.1.1.5. Give an example to trace-by-hand the modified algorithm. What is its corresponding state space tree?



6.1.2. The Eight Queens problem

- ❑ This problem was investigated by C.F. Gauss in 1850, but he did not completely solve it.
- ❑ The eight queens problem is stated as:

“Eight queens are to be placed on a chess board in such a way that no queen checks against any other queens”.
- ❑ Chess rule: *A queen can attack all other queens in either the same column, row, or diagonals on the board.*
- ❑ Using the backtracking template, we obtain an initial version of a *backtracking* algorithm for the 8 queens problem.



6.1.2. The Eight Queens problem

One
solution for
the *Eight
Queens*
problem

1	Q1							
2							Q7	
3					Q5			
4								Q8
5		Q2						
6				Q4				
7						Q6		
8			Q3					



6.1.2. The Eight Queens problem

```
procedure try (i: integer);  
begin  
  initialize selection of positions for the i-th queen;  
  repeat  
    make next selection;  
    if safe then  
      begin  
        setqueen;  
        if  $i < 8$  then  
          begin  
            try ( $i + 1$ );  
            if not successful then remove queen  
          end  
        end  
      until successful  $\vee$  no more positions  
    end
```



6.1.2. The Eight Queens problem

Data representation

How to represent 8 queens on the board?

```
var x: array[1..8] of integer;  
    a: array[1..8] of Boolean;  
    b: array[b1..b2] of Boolean;  
    c: array[c1..c2] of Boolean;
```

where

$x[i]$ denotes the row position of the queen in the i column.

$a[j]$ means no queen lies in the j -th row.

$b[k]$ means no queen occupies the k -th $\swarrow$ diagonal.

$c[k]$ means no queen sits on the k -th $\searrow$ diagonal.



6.1.2. The Eight Queens problem

The choice for index bounds $b1$, $b2$, $c1$, and $c2$ is dictated by the way that indices of b and c are computed. We note that in the $\swarrow$ diagonal all fields have the same sums of their coordinates $i + j$, and that in the $\searrow$ diagonal, the coordinate differences $(i - j)$ are constant.

Given these data, the statement *setqueen* is refined as:

$$x[i] := j; a[j] := \text{false}; b[i+j] := \text{false}; c[i-j] := \text{false};$$

The statement *removequeen* is refined as:

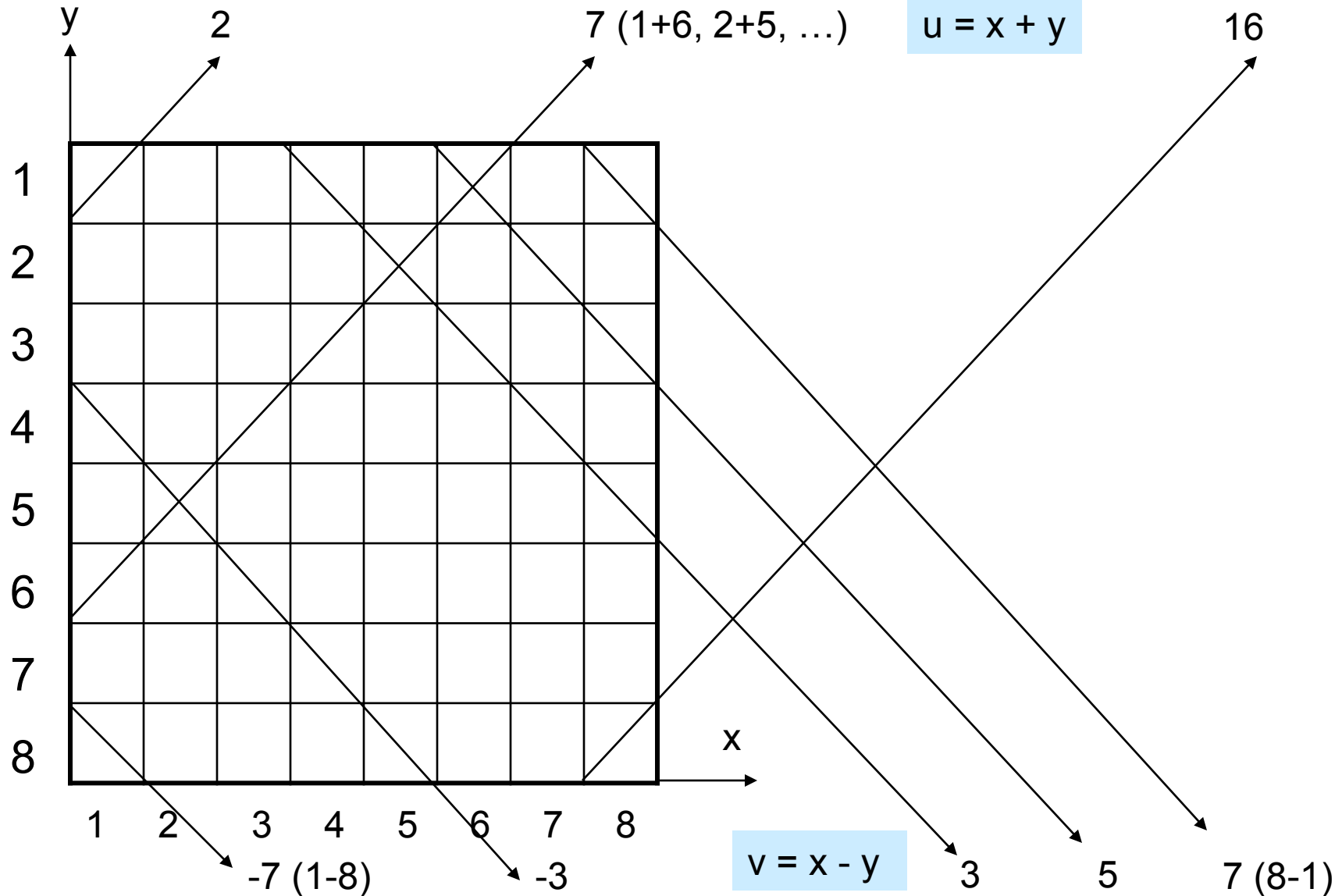
$$a[j] := \text{true}; b[i+j] := \text{true}; c[i-j] := \text{true};$$

The condition *safe* is represented as:

$$a[j] \wedge b[i+j] \wedge c[i-j]$$



6.1.2. The Eight Queens problem





6.1.2. The Eight Queens problem

```
program eightqueen1(output);  
  {find one solution to eight queens  
  problem}  
var      i : integer; q: boolean;  
a : array [1..8] of boolean;  
b : array [2..16] of boolean;  
c : array [-7..7] of boolean;  
x : array [1..8] of integer;  
procedure try(i: integer; var q:  
boolean);  
var j: integer;  
begin  
  j:=0;  
  repeat  
    j:=j+1; q:=false;  
    if a[j]  $\wedge$  b[i+j]  $\wedge$  c[i-j] then
```

```
    begin  
      x[i]:=j;  
      a[j]:=false; b[i+j]:=false;  
      c[i-j]:=false;  
      if i<8 then  
        begin  
          try (i+1, q);  
          if  $\neg$  q then  
            begin  
              a[j]:=true; b[i+j]:=true;  
              c[i-j]:=true  
            end  
          end  
          else q:=true  
        end  
      until q  $\vee$  (j=8)  
    end {try};
```



6.1.2. The Eight Queens problem

Time complexity of
a *backtracking*
algorithm for the
eight queens
problem:

- Abstract operation:
placement
- *Worst case* analysis:
 $C_n = O(n!)$

```
program eightqueen1(output);  
{find one solution to eight queens problem}  
...  
begin  
  for i:= 1 to 8 do a[i]:=true;  
  for i:= 2 to 16 do b[i]:=true;  
  for i:= -7 to 7 do c[i]:=true;  
  try (1,q);  
  if q then  
    for i:=1 to 8 do  
      write (x[i]:4);  
  writeln  
end
```



6.1.2. The Eight Queens problem

One
solution for
the *Eight
Queens*
problem

1	Q1							
2							Q7	
3					Q5			
4								Q8
5		Q2						
6				Q4				
7						Q6		
8			Q3					



6.1.2. The Eight Queens problem

- A solution to the four queens problem

1	Q1			
2				
3				
4				

1	Q1			
2				
3		Q2		
4				

Q3
X

1	Q1			
2				
3		Q2		
4				

1	Q1			
2				
3				
4		Q2		



6.1.2. The Eight Queens problem

1	Q1			
2				
3				
4		Q2		

1	Q1			
2			Q3	
3				
4		Q2		

1	Q1			
2			Q3	
3				
4		Q2		

H4
X

1			Q3	
2	Q1			
3				Q4
4		Q2		



6.1.2. The Eight Queens problem

- Extension for all the solutions
 - The extension is to find not only one but all the solutions to the posed problem.
 - Extension method: *Once a solution is found and recorded, we proceed to the next candidate delivered by the systematic selection process.*
 - The general schema is derived from the backtracking template and modified to generate all the solutions.



6.1.2. The Eight Queens problem

A *backtracking* algorithm template to generate *all the solutions* to a given problem

```
procedure try(i: integer);  
var k: integer;  
begin  
  for k:=1 to m do  
    begin  
      select the k-th candidate;  
      if acceptable then  
        begin  
          record it;  
          if i<n then try (i+1) else print solution;  
          cancel recording  
        end  
      end  
    end  
  end
```



6.1.2. The Eight Queens problem

In the extended algorithm, to simplify the stopping condition of the selection process, the *repeat* statement is replaced by a *for* statement.

```
program eightqueens(output);  
var i: integer;  
a: array [1.. 8] of boolean;  
b: array [2.. 16] of boolean;  
c: array [-7.. 7] of boolean;  
x: array [1.. 8] of integer;  
procedure print;  
var k : integer;  
begin  
    for k := 1 to 8 do write(x[k]:4);  
    writeln  
end {print};
```

```
procedure try (i:integer);  
var j: integer;  
begin  
    for j:=1 to 8 do  
        if a[j]  $\wedge$  b[i+j]  $\wedge$  c[i-j] then  
            begin  
                x[i]:=j;  
                a[j]:=false; b[i+j]:= false;  
                c[i-j]:=false;  
                if i < 8 then try(i+1) else print;  
                a[j]:=true; b[i+j]:= true;  
                c[i-j]:= true;  
            end  
end {try};
```



6.1.2. The Eight Queens problem

```
program eightqueens(output);  
...  
begin  
  for i:= 1 to 8 do a[i]:=true;  
  for i:= 2 to 16 do b[i]:=true;  
  for i:= -7 to 7 do c[i]:=true;  
  try(1);  
end.
```

The extended algorithm gives all 92 solutions for the 8 queens problem.

Actually, there are only 12 significantly different solutions.



6.1.2. The Eight Queens problem

The 12 solutions generated are listed in the following table:

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	N
1	5	8	6	3	7	2	4	876
1	6	8	3	7	4	2	5	264
1	7	4	6	8	2	5	3	200
1	7	5	8	2	4	6	3	136
2	4	6	8	3	1	7	5	504
2	5	7	1	3	8	6	4	400
2	5	7	4	1	8	6	3	72
2	6	1	7	4	8	3	5	280
2	6	8	3	1	4	7	5	240
2	7	3	6	8	5	1	4	264
2	7	5	8	1	4	6	3	160
2	8	6	1	3	5	7	4	336

The numbers N to the right indicate the frequency of execution of the test for safe fields. Its average over all 92 solutions is 161.



6.1.2. The Eight Queens problem

Your turn:

- ❑ 6.1.2.1. Draw a state space tree showing a solution to the *four queens* problem.
- ❑ 6.1.2.2. Trace-by-hand the backtracking algorithm for the five queens problem. How many solutions are there for the *five queens* problem. List all the solutions.
- ❑ 6.1.2.3. Trace-by-hand the backtracking algorithm for the *six queens* problem. How many solutions are there for the *six queens* problem.



6.2. Branch-and-Bound algorithms

Branch-and-bound algorithms:

- ❑ construct a solution *one component at a time*, trying to terminate the process as soon as one can ascertain that no solution can be obtained as a result of the choices already made.
- ❑ employ, as a principal mechanism, a *state space tree*, a rooted tree whose nodes represent partially constructed solutions to the problem in question.
- ❑ *terminate a node as soon as it can be guaranteed that no solution* to the problem can be obtained by considering choices that correspond to the node's descendants.
- ❑ enhance the idea of generating a state space tree with the idea of *estimating the best value obtainable from a current node* of the decision tree: if such an estimate is not superior to the best solution seen up to that point in the processing, the node is eliminated from further consideration.



6.2. Branch-and-Bound algorithms

- ❑ Branch-and-bound is applicable only to *optimization problems* because it is based on computing a *bound* on possible values of the problem's *objective function*.
 - A bound should be a lower bound for a minimization problem and an upper bound for a maximization problem.
 - ❑ Branch-and-bound requires:
 - a way to provide, *for every node* of a state space tree, *a bound on the best value of the objective function on any solution* that can be obtained by adding further components to the partially constructed solution represented by the node
 - the *value of the best solution seen so far*
- ➔ *Principal idea of branch-and-bound*: a node's bound value is *compared* with the value of the best solution seen so far: if the bound value is not better than the best solution seen so far, the node is *nonpromising* and can be terminated (the *branch is "pruned"*) because no solution obtained from it can yield a better solution than the one already available.



6.2. Branch-and-Bound algorithms

Your turn:

- ❑ 6.2.1. List 3 main characteristics of a branch-and-bound algorithm.
- ❑ 6.2.2. Compare the branch-and-bound strategy with the backtracking one.
- ❑ 6.2.3. Given an n -by- n cost matrix where a value at cell (i, j) shows a cost of person i handling job j , the assignment problem is to assign n people to n jobs so that the total cost of the assignment is as small as possible. Describe how to apply branch-and-bound on this problem.



6.2.1. Traveling Salesman problem

- ❑ **Traveling Salesman Problem (TSP):** Given a set of N cities, find the shortest route connecting them all, with no city visited twice.
- ❑ This brings to mind another problem: given an undirected graph, is there any way to connect all the nodes with a simple cycle? This is known as **Hamiltonian Cycle Problem (HCP)**.
- ❑ To solve HCP, we can modify the depth-first-search (DFS) algorithm to generate *all simple paths which visit all vertices in the graph*.



6.2.1. Traveling Salesman problem

- Exhaustive search: Modified DFS to generate all the **simple paths**

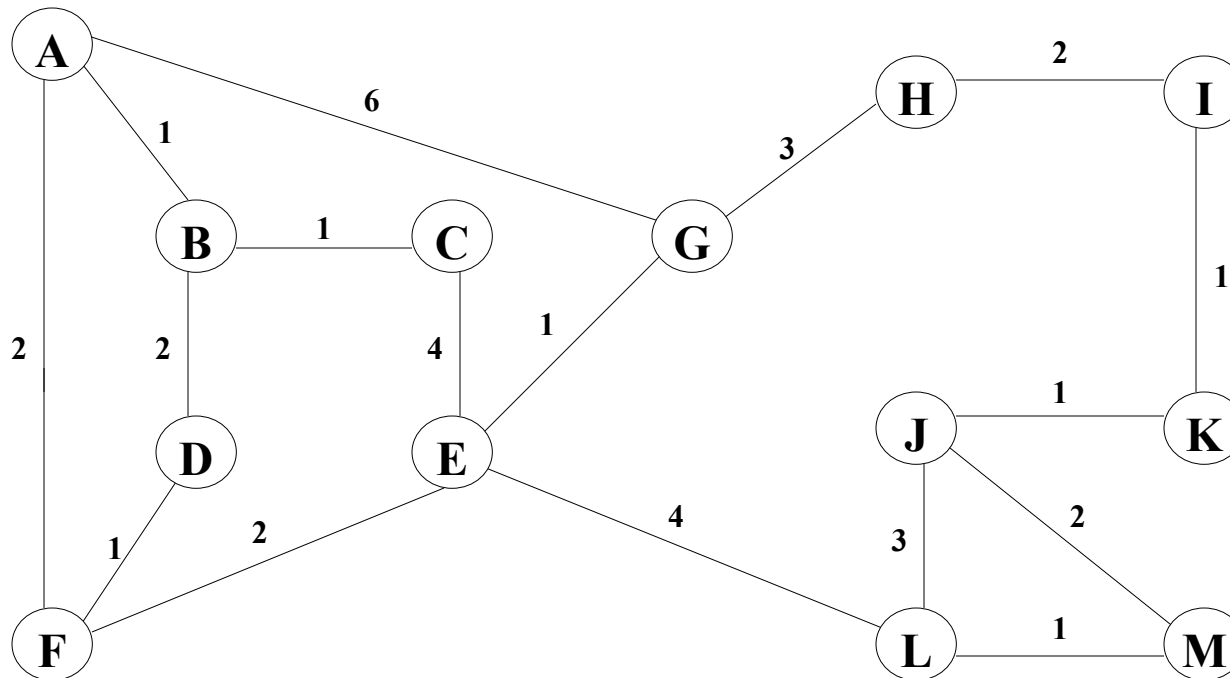
```
procedure visit( k: integer);  
var t: integer;  
begin  
    id := id + 1; val[k]:= id;  
    for t:= 1 to V do  
        if a[k, t] then  
            if val[t]= 0 then visit(t);  
    id := id - 1; val[k] := 0  
end;
```

This recursive procedure can generate all simple paths from a starting vertex:

```
id := 0;  
for k:= 1 to V do val[k]:=0;  
    visit(1);
```

6.2.1. Traveling Salesman problem

□ Given a following graph





- [illegible]

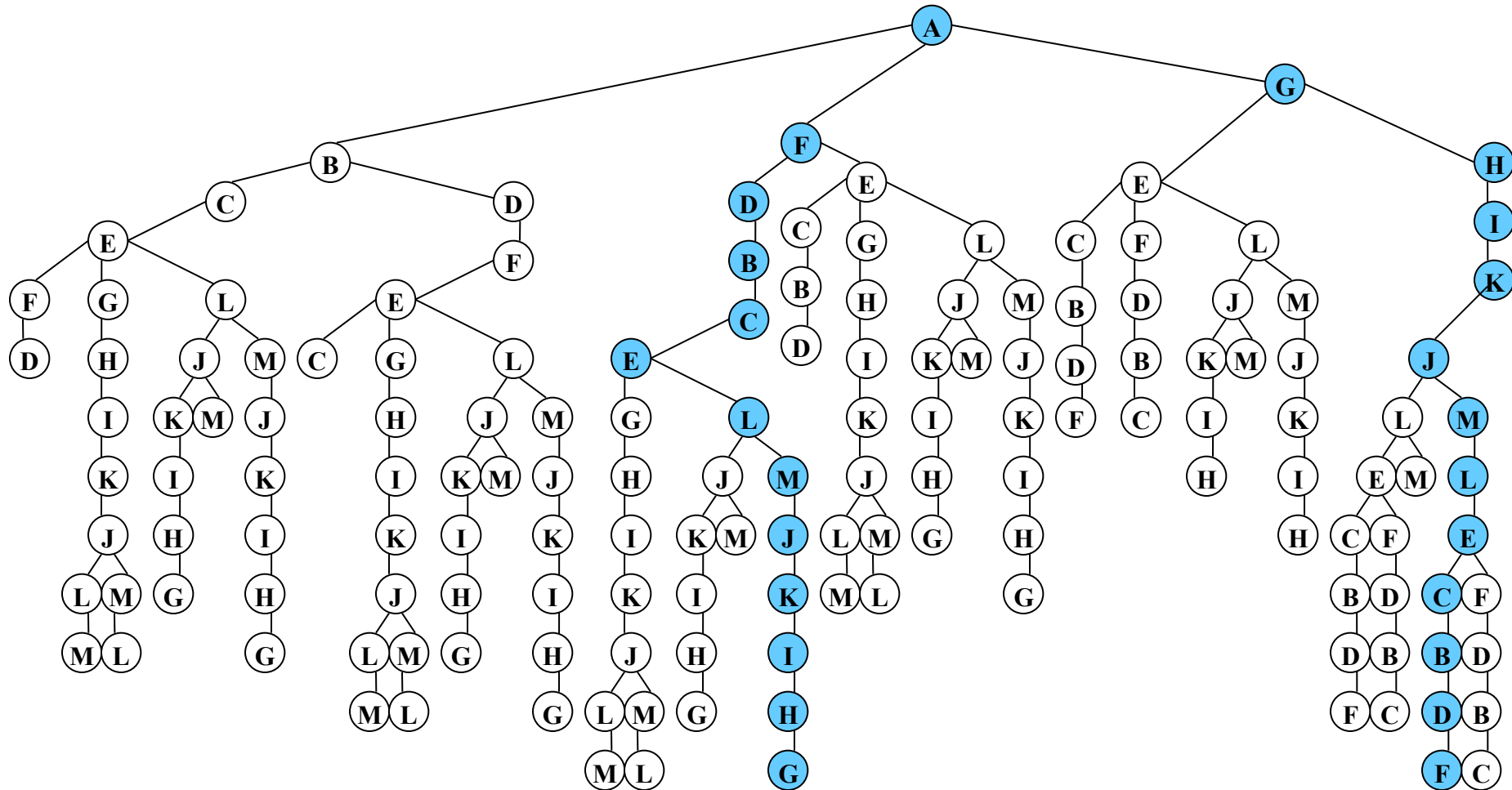


6.2.1. Traveling Salesman problem

- From the algorithm generating all the simple paths to the algorithm solving TSP
 - We can make the *visit* procedure above test for the existence of Hamiltonian cycles by having it test whether there is an edge from k to 1 when $val[k]=V$.
 - In the example above, there are 2 Hamiltonian cycles which are in fact the same.
 - A F D B C E L M J K I H G
 - A G H I K J M L E C B D F
 - The algorithm for finding Hamiltonian cycles can be modified to solve the TSP problem by keeping track of the cost of the current path and searching for the path with the smallest cost among the found Hamiltonian cycles.
- This is designed with *branch-and-bound*.

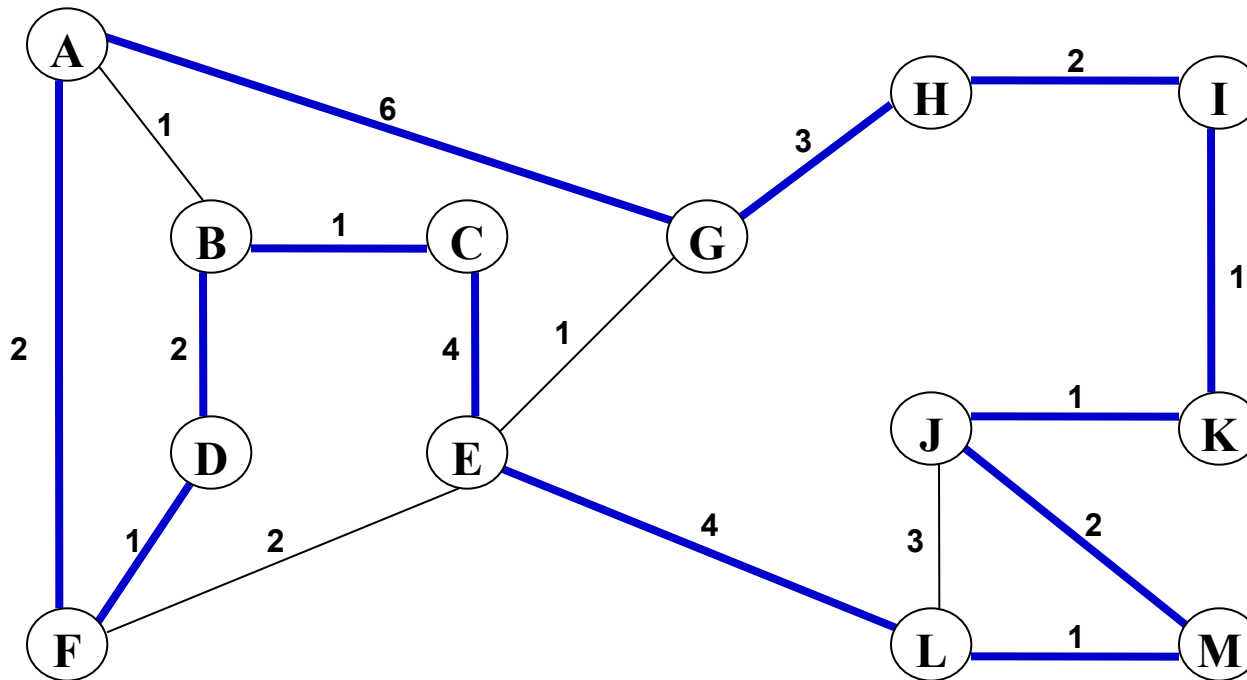
6.2.1. Traveling Salesman problem

- Two Hamiltonian cycles in the given graph



6.2.1. Traveling Salesman problem

- The single Hamiltonian cycle in the graph





6.2.1. Traveling Salesman problem

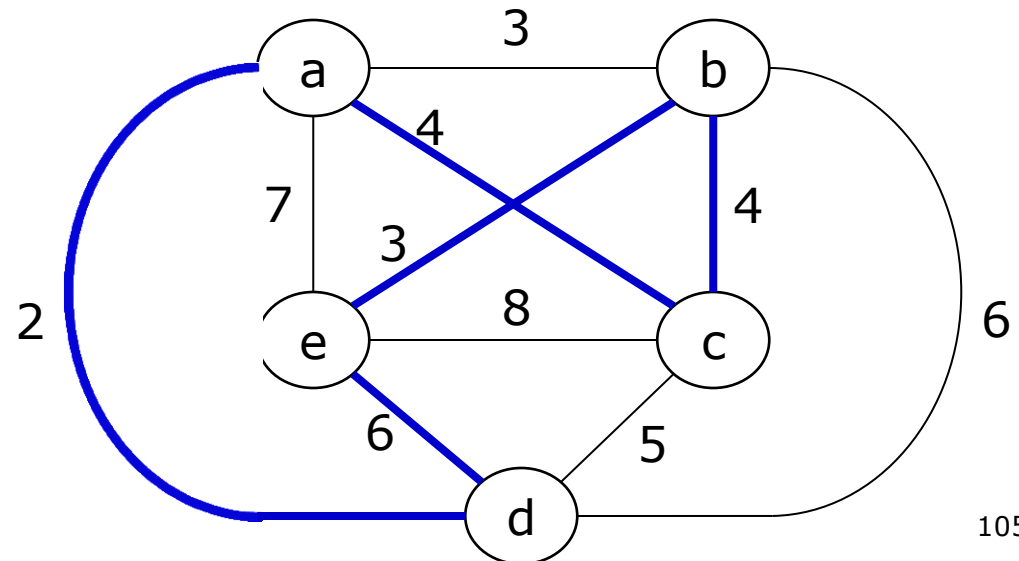
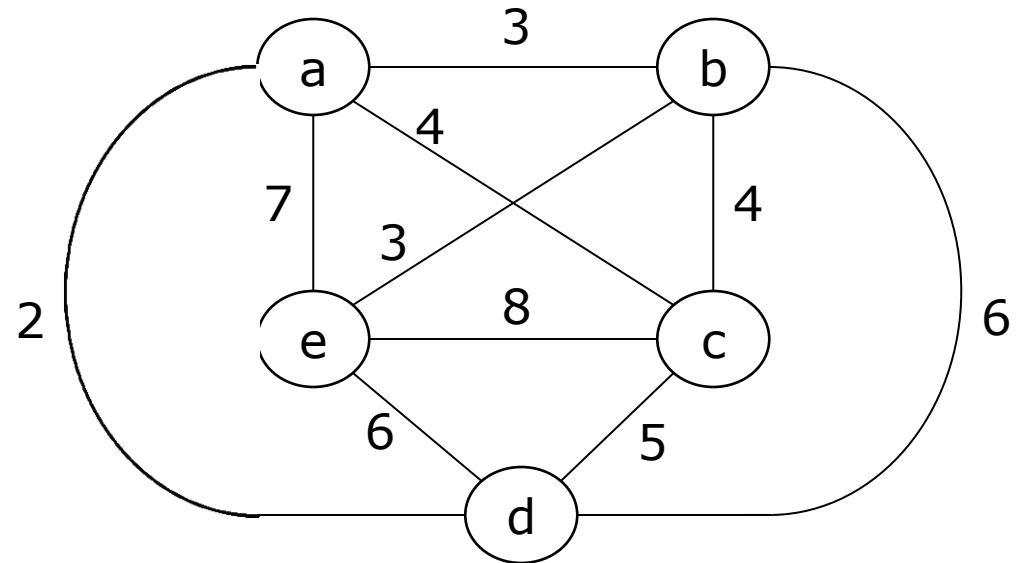
- Branch-and-bound for solving the traveling salesman problem
 - When applying a modified DFS algorithm to generate all simple paths, in searching the best path for TSP problem, there is an important pruning technique: we *terminate the search as soon as it is determined that it can't possibly be successful*.
 - Suppose a path of cost x through the graph has been found. Then it is fruitless to continue along any path for which the cost so far is *greater than* x . This can be implemented simply by making no recursive calls in *visit* if the cost of the current *partial path* is greater than *the cost of the best full path* found so far.

6.2.1. Traveling Salesman problem

- Solve the traveling salesman problem with the given graph

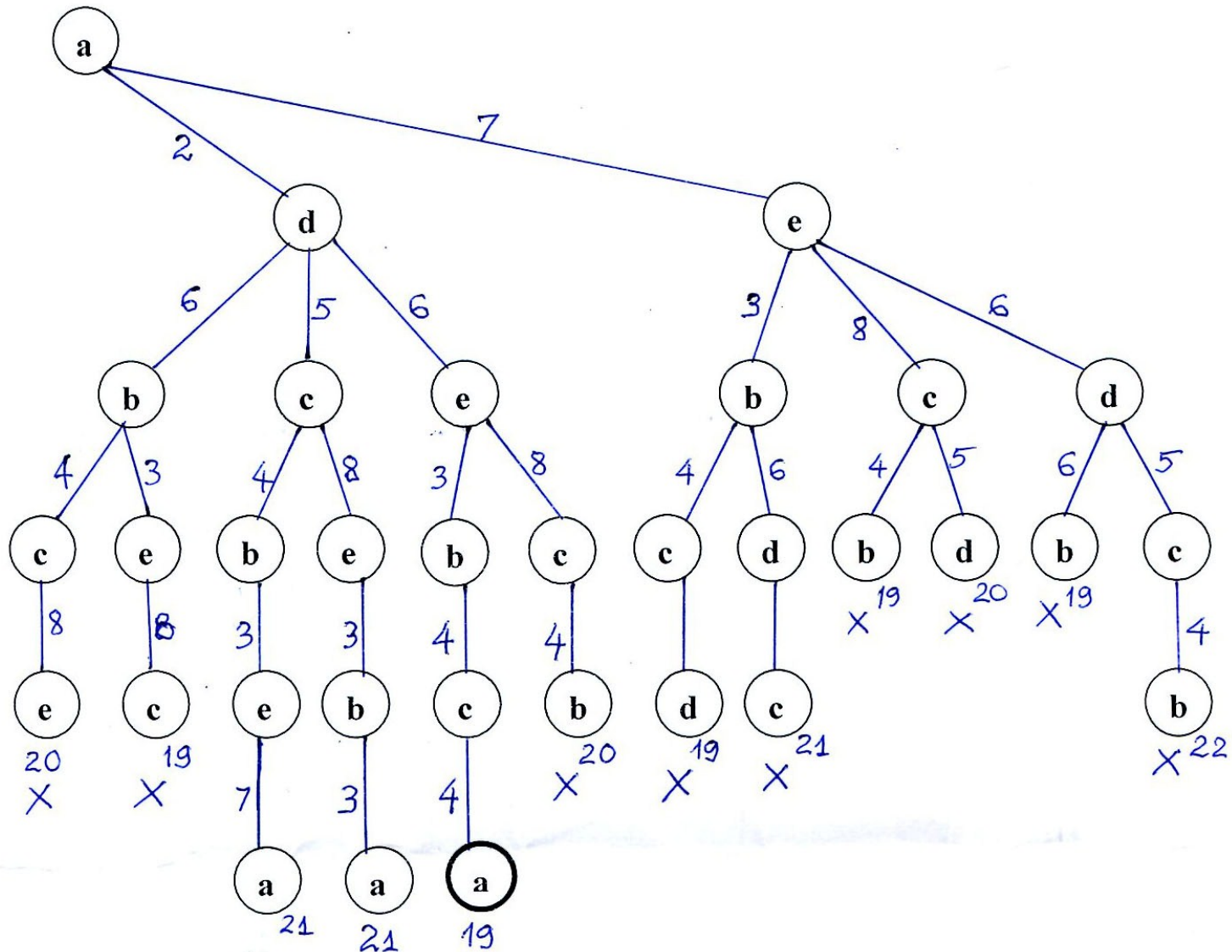
- Tour: **a c b e d a**

- Minimum cost: **19**





6.2.1. Traveling Salesman problem





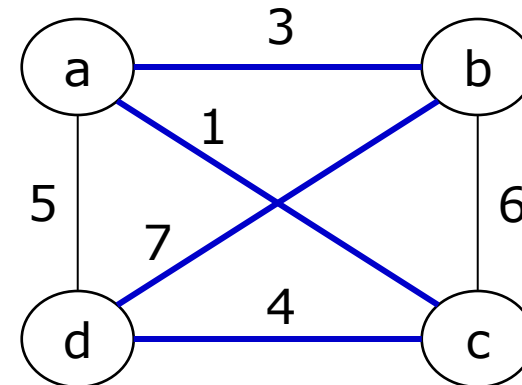
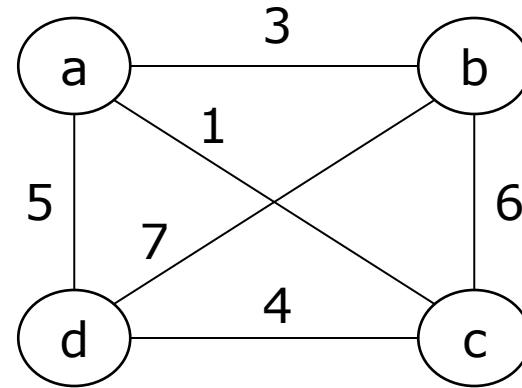
6.2.1. Traveling Salesman problem

- ❑ Combine DFS and lower bound to derive the tour and the minimum cost of the traveling salesman problem
- ❑ *A lower bound lb on the length l of any tour is computed as:*
 - *For each city i , $i=1..n$, find the sums s_i of the distances from city i to the two nearest cities; compute the sum s of these n numbers; divide the result by 2; and, if all the distances are integers, round up the result to the nearest integer: $lb = \lceil s/2 \rceil$*
 - *lb for each visited city is updated with the cost of the required edge to that city.*

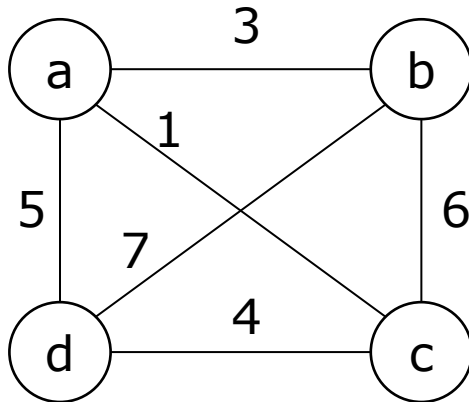


6.2.1. Traveling Salesman problem

- Solve the traveling salesman problem with the given graph
 - Tour: **a b d c a**
 - Minimum cost: **15**



6.2.1. Traveling Salesman problem



lb at the starting vertex $a =$

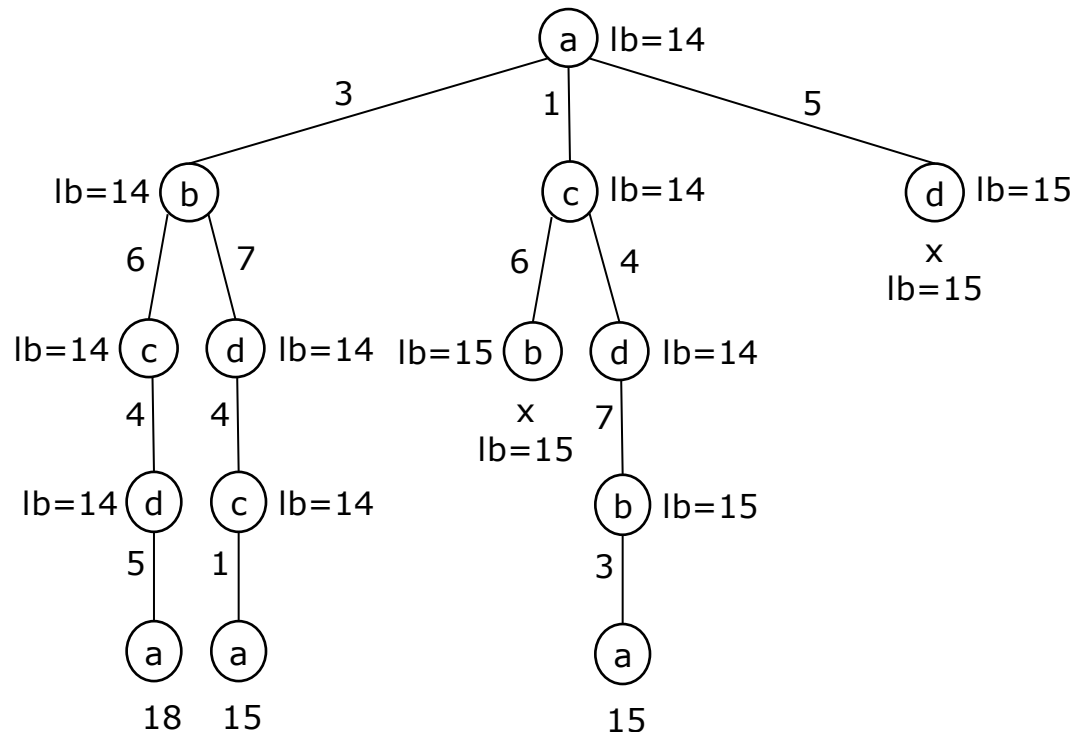
$$lb \text{ at } a = \lceil ((1+3)+(3+6)+(1+4)+(4+5))/2 \rceil = 14$$

From a to b : *lb* at $b = lb$ at $a = 14$ because cost on edge (a,b) has already been included in *lb*.

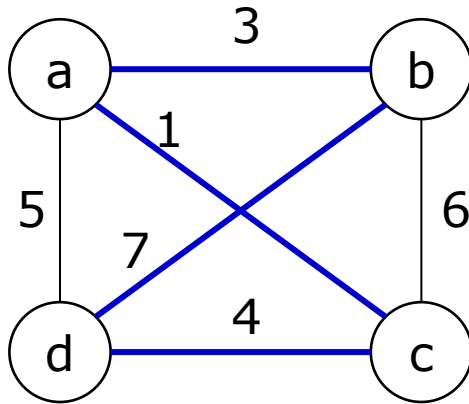
From a to b to d : *lb* at d is updated by replacing 6 with 7 to include the cost on edge (b,d) .

$$lb \text{ at } d = \lceil ((1+3)+(3+7)+(1+4)+(4+5))/2 \rceil = 14$$

$$\text{From } a \text{ to } d: lb \text{ at } d = \lceil ((1+5)+(3+6)+(1+4)+(4+5))/2 \rceil = 15$$



6.2.1. Traveling Salesman problem



lb at the starting vertex $a =$

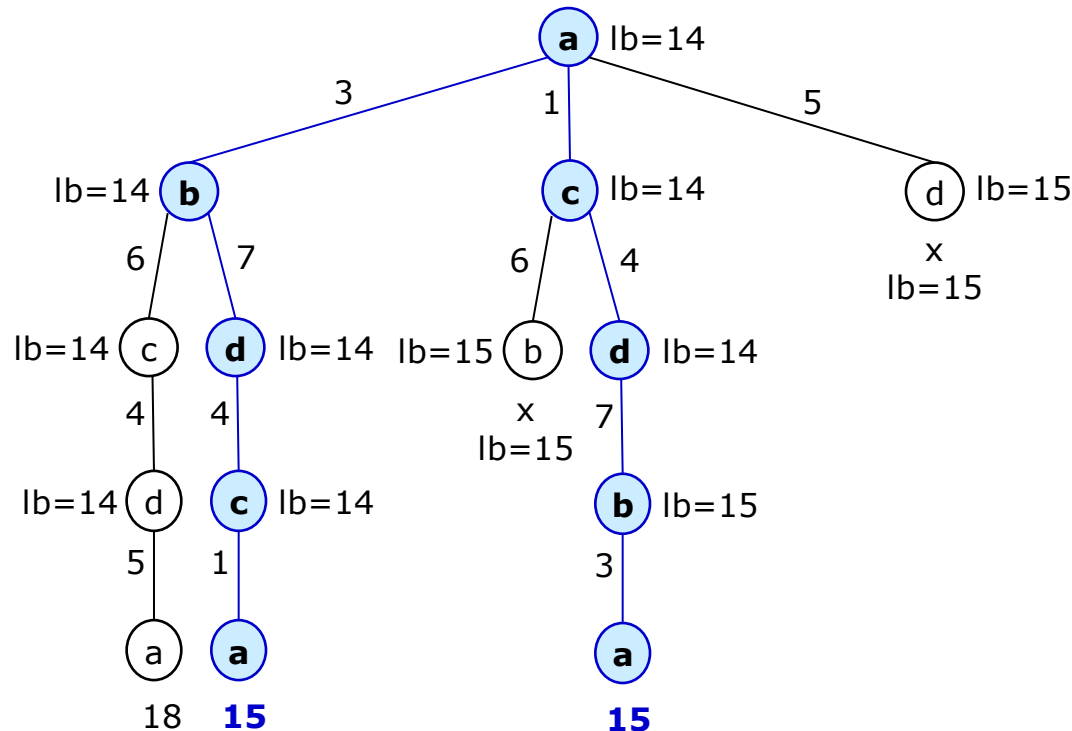
$$lb \text{ at } a = \lceil ((1+3)+(3+6)+(1+4)+(4+5))/2 \rceil = 14$$

From a to b : *lb* at $b = lb$ at $a = 14$ because cost on edge (a,b) has already been included in *lb*.

From a to b to d : *lb* at d is updated by replacing 6 with 7 to include the cost on edge (b,d) .

$$lb \text{ at } d = \lceil ((1+3)+(3+7)+(1+4)+(4+5))/2 \rceil = 14$$

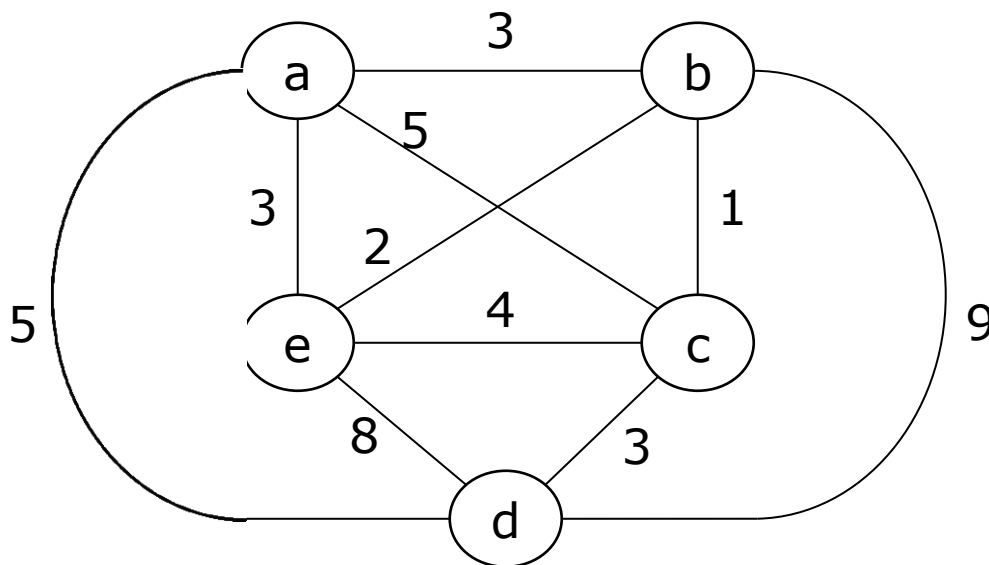
$$\text{From } a \text{ to } d: lb \text{ at } d = \lceil ((1+5)+(3+6)+(1+4)+(4+5))/2 \rceil = 15$$



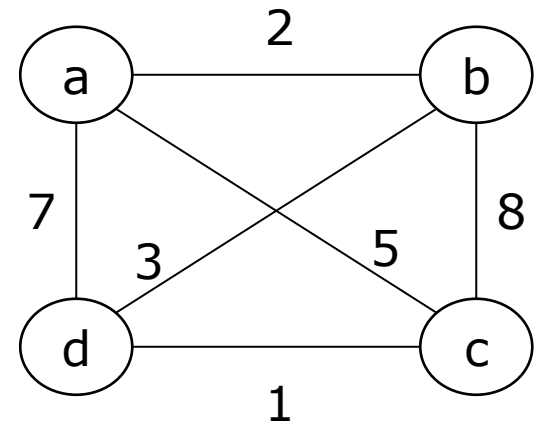
6.2.1. Traveling Salesman problem

Your turn:

- Solve the traveling salesman problem with each following graph. What is the tour? How about its minimum cost?



6.2.1.1.

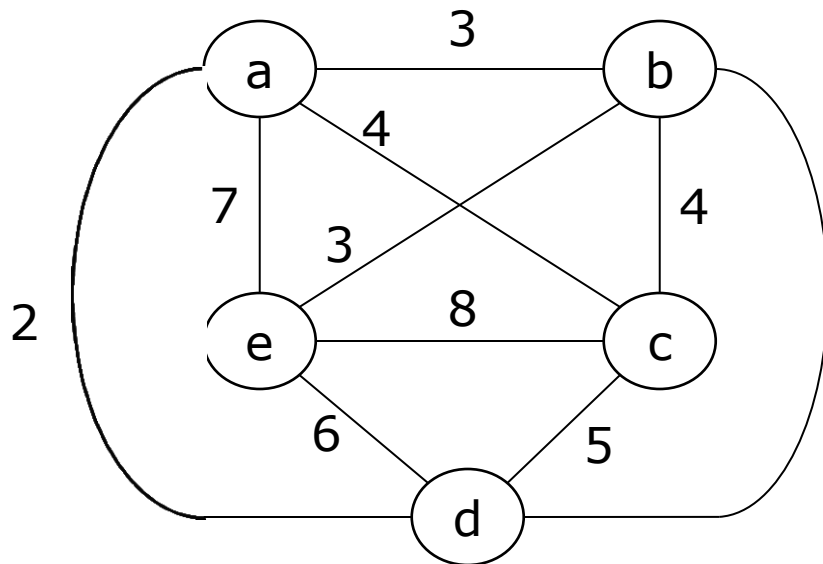


6.2.1.2.

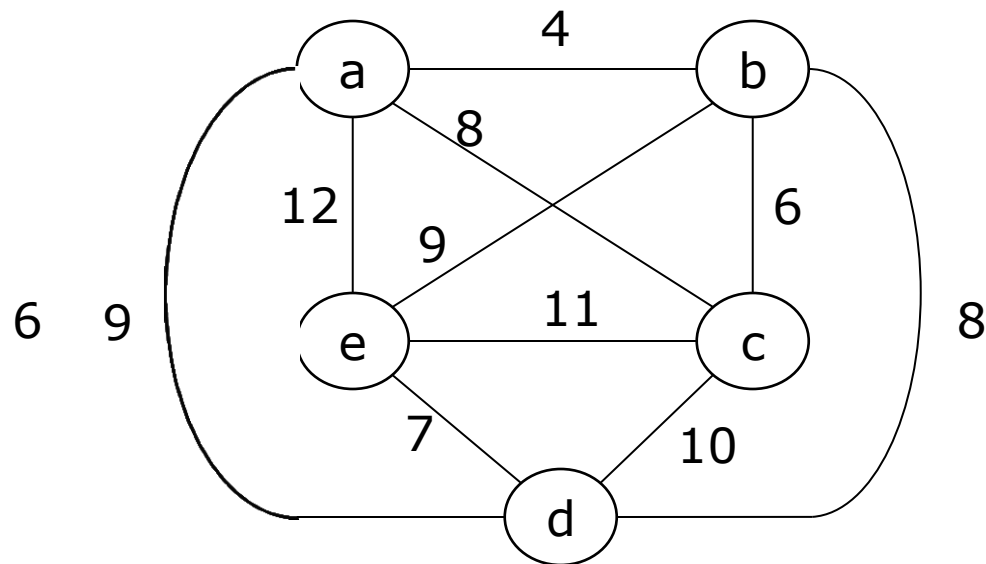
6.2.1. Traveling Salesman problem

Your turn:

- Solve the traveling salesman problem with each following graph. What is the tour? How about its minimum cost?



6.2.1.3



6.2.1.4



6.2.1. Traveling Salesman problem

Your turn:

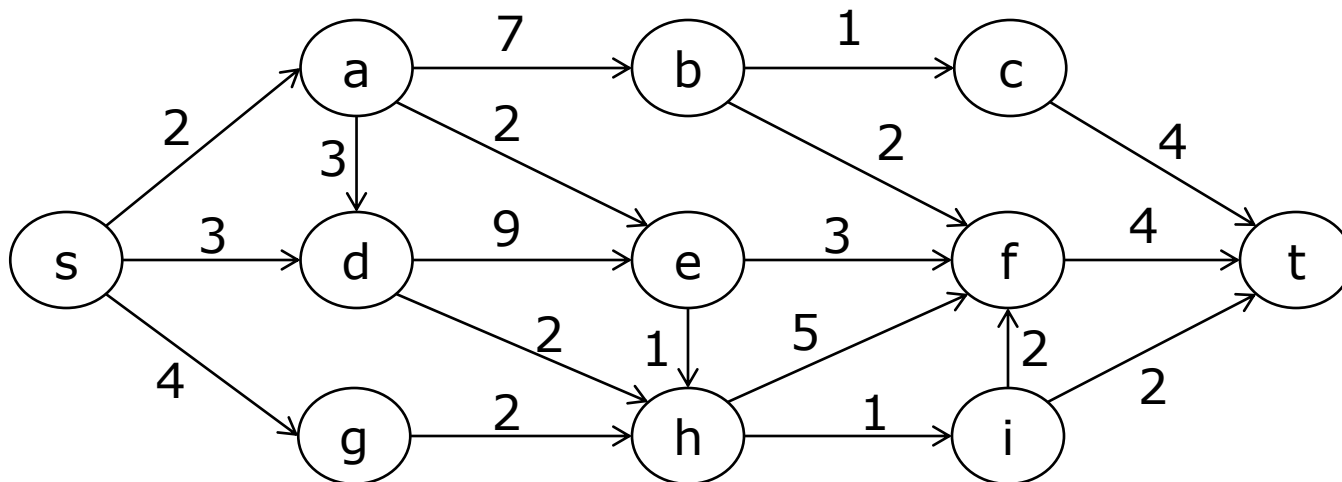
- ❑ 6.2.1.5. Design and analyze a branch-and-bound algorithm for the *assignment problem*. A lower bound on the cost of an optimal selection can be defined as the sum of the selected ones and the smallest elements in each of the cost matrix's rows for the not yet processed ones.
- ❑ 6.2.1.6. Given a cost matrix for 4 people and 4 jobs. What is the *most cost-effective assignment solution*? Show its *state space tree*.

$$C = \begin{matrix} & \begin{matrix} \text{job 1} & 2 & 3 & 4 \end{matrix} \\ \begin{bmatrix} 3 & 6 & 2 & 5 \\ 4 & 2 & 8 & 1 \\ 7 & 3 & 4 & 2 \\ 5 & 9 & 4 & 3 \end{bmatrix} & \begin{matrix} \text{person 1} \\ \text{person 2} \\ \text{person 3} \\ \text{person 4} \end{matrix} \end{matrix}$$

6.2.1. Traveling Salesman problem

Your turn:

- 6.2.1.7. Design and analyze a branch-and-bound algorithm to find the shortest path from vertex s to vertex t in a weighted directed graph. A lower bound can be defined as a total weight sum of the weights along the path from vertex s to the current one.
- 6.2.1.8. Given a weighted directed graph as follows, what is the shortest path from s to t ? How about its minimum weight? Show its state space tree.



Algorithms Analysis and Design (C03031)

- 6.1. Backtracking algorithms
 - 6.1.1. The Knight's Tour problem
 - 6.1.2. The Eight Queens problem
- 6.2. Branch-and-Bound algorithms
 - 6.2.1. Traveling Salesman problem

